

universität freiburg

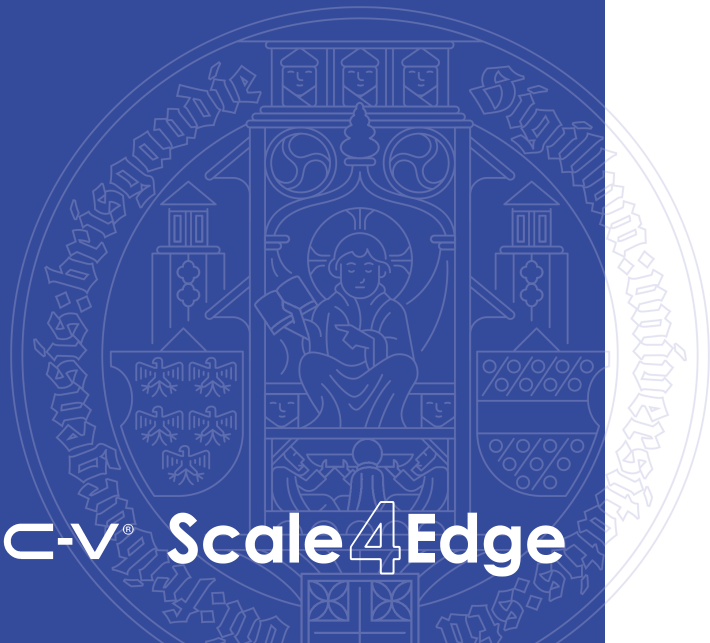
Test and Reliability

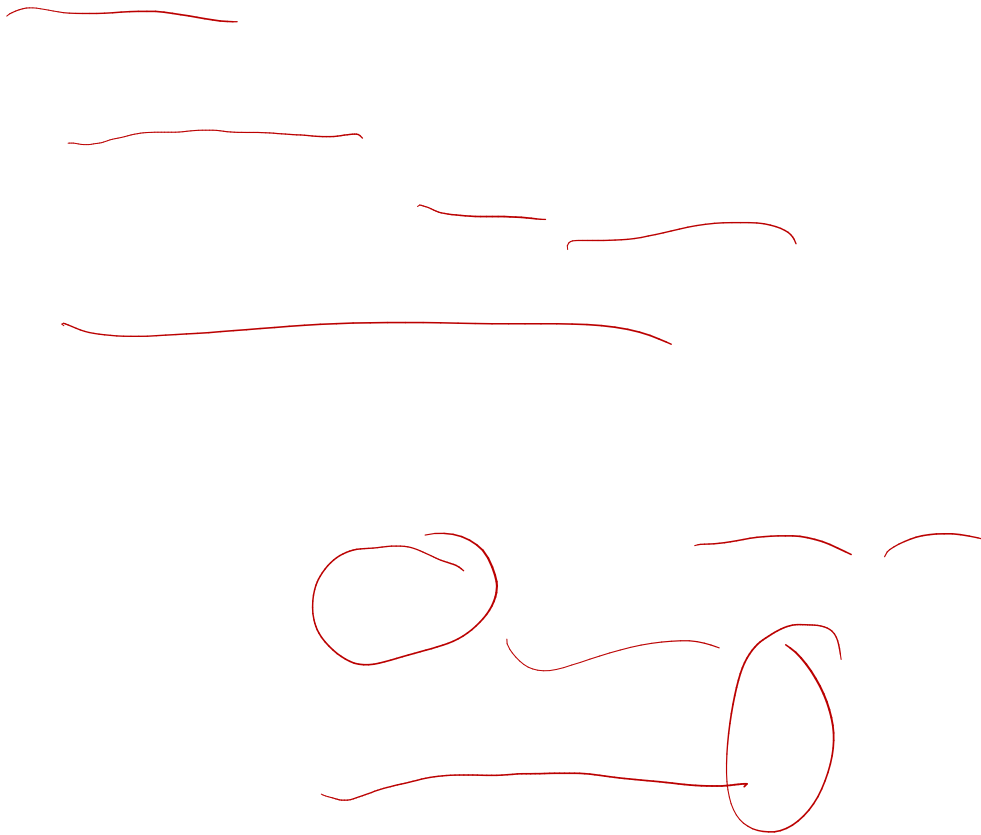
Chapter 9: Cell-Aware Testing

Tobias Faller, M.Sc.
Prof. Dr. Ralf Wimmer
Dr. Tobias Seufert

Summer Term 2026

 **RISC-V®** **Scale4Edge**





Motivation

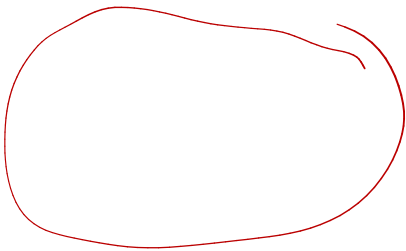
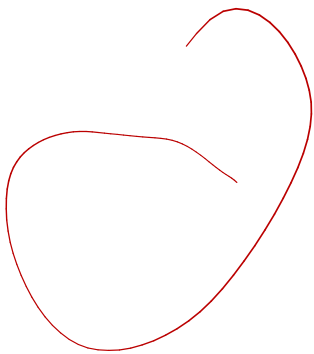
Cell-Aware Testing

- Miniturization of integrated circuits is progressing
 - Increased susceptibility to defects
 - Traditional fault models (stuck-at / transition-delay) are increasingly inadequate

⇒ Demands for testing exceeds capability
- Defective chips are undetected and delivered to consumers / clients
 - Faults show up as field returns (failing systems)
 - Faults show up as Silent Data Corruption (SDC) [1], [2]
- New testing approach that better models physical defects is required

Motivation - References I

- [1] Peter H. Hochschild, Paul Jack Turner, Jeffrey C. Mogul, et al. “Cores that don't count”. In: *Proc. 18th Workshop on Hot Topics in Operating Systems (HotOS 2021)*. 2021.
- [2] Harish Dattatraya Dixit, Sneha Pendharkar, Matt Beadon, et al. *Silent Data Corruptions at Scale*. _eprint: 2102.11245. 2021. URL: <https://arxiv.org/abs/2102.11245>.



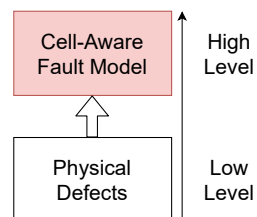
Outline

- Cell-Aware Testing
Example: Cell-Aware SBST Generation for RISC-V CLIC (Scale4Edge)
- Cell-Aware Fault Models
Example: Cell-Aware Fault Model Generation (Scale4Edge)

Cell-Aware Testing

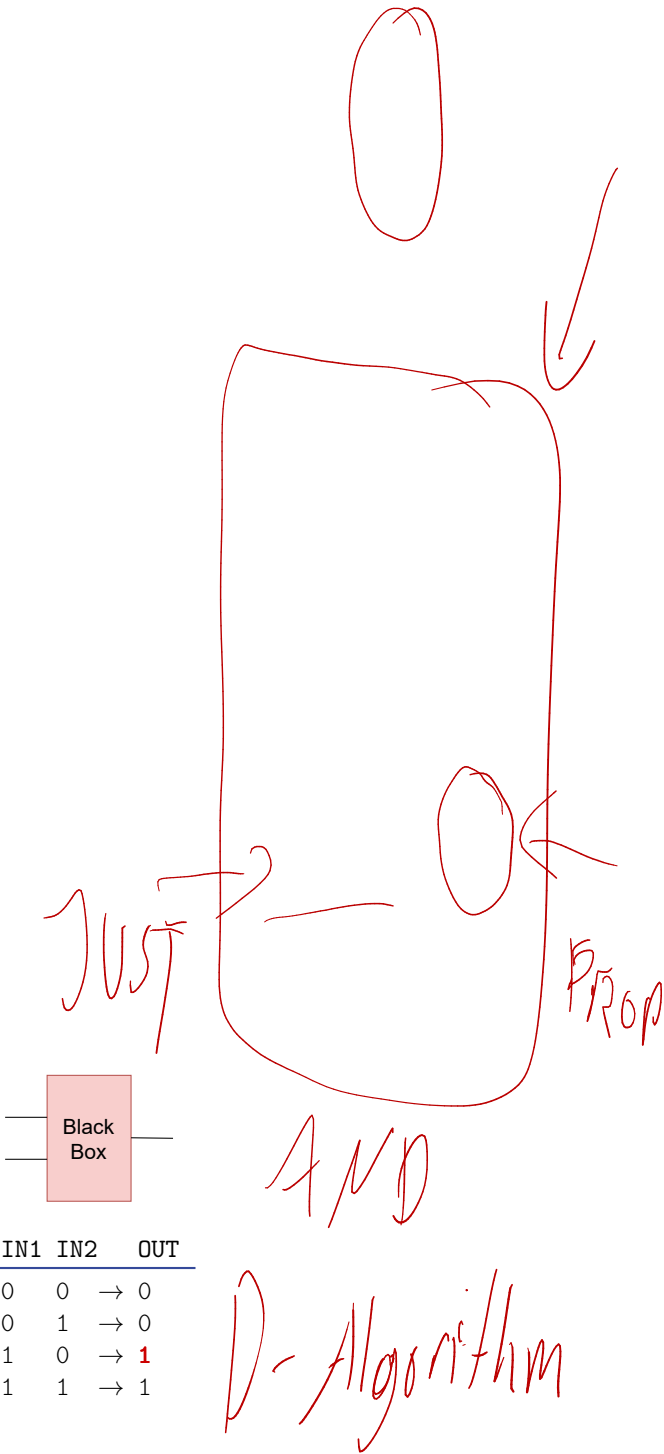
Cell-Aware Testing (CAT) [1]-[3]

- Adaptable fault model specific to ...
 - production process
 - cell library
 - test type (production test, binning, burn-in)
- Test only for defects that are relevant
 - Use occurring physical (low-level) defects as basis for test patterns
 - Derive cell-aware fault model (high-level) from these defects
 - (Extend fault model based on yield and field return data)

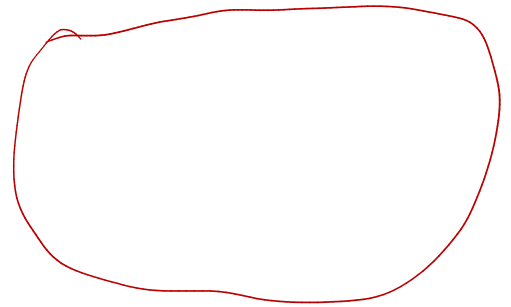
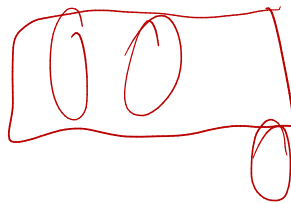


Cell-Aware Testing

- ATPG replaces faulty cell with cell-aware fault model
 - Fault model specifies cell behavior under defect influence
 - Fault effects are encoded in fault model
- ATPG applies cell-aware fault model specification to black box
 1. Inputs to be justified on cell inputs
 2. Outputs to be propagated from cell outputs



1 0 → X



Cell-Aware Testing

User-Defined Fault-Model (UDFM) [4]

Cell "AND"

Fault "bridge T1:bulk to T2:drain"

Static Test

Transition Test

Transition Test

Fault "open T6:drain"

Transition Test

IN1	IN2	OUT
1	1	→ 0
01	00	→ 01
00	01	→ 01

IN1	IN2	OUT
11	10	→ 11

Cell-Test Model (CTM) [5]

• Cell: AND

• Faults:

F1) bridge T1:bulk to T2:drain

F2) open T6:drain

IN1	IN2	OUT	F1	F2
1	1	→ 1	1	0
R	0	→ 0	1	0
0	R	→ 0	1	0
1	F	→ F	0	1

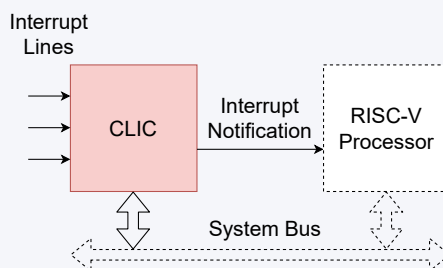
Cell-Aware Testing

Cell-Aware SBST Generation for RISC-V CLIC

Automatic Cell-Aware Software-Based Self-Test Generation for a RISC-V Core Local Interrupt Controller

IEEE ATS 2025 – Paper

- SBST for RISC-V Core Local Interrupt Controller
 - Constrained ATPG with Validity Checker Module (VCM)
 - Constraints on CLIC memory bus interface
 - SBST instructions from bus transactions
- Cell-aware fault model for functional test generation
 - Adapt cell-aware testing to functional test
 - Compare coverage with stuck-at fault model



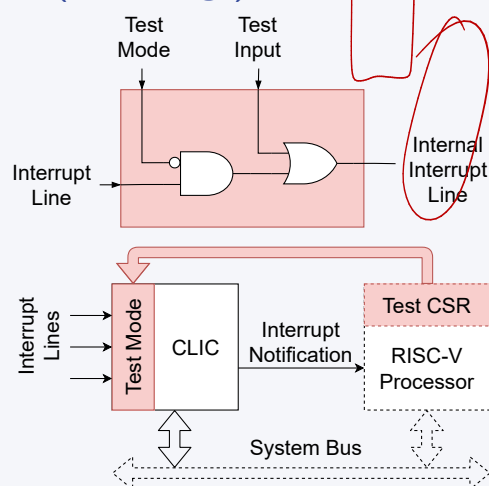
Scale4Edge
RISC-V®

Cell-Aware Testing

Cell-Aware SBST Generation for RISC-V CLIC (Scale4Edge)

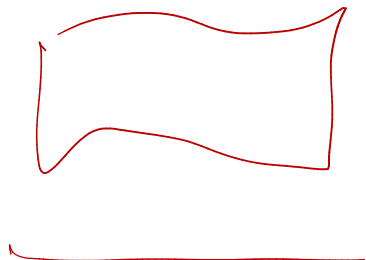
Testing generation strategy:

- Include test interface to control interrupts
- Control test interface via processor CSR
- Constraints include handling of test interface
 - Only either bus transaction, interrupt or CSR access
 - Interrupts are acknowledged after n cycles
 - Bus transactions follow bus protocol



0 1
X - [] - A₁
1 - []

01/x

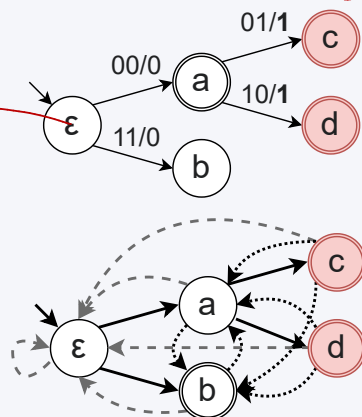


Cell-Aware Testing

Cell-Aware SBST Generation for RISC-V CLIC (Scale4Edge)

Functional cell-aware fault model:

- Use state machine instead of static test table
 - Cell-aware tests from a deterministic prefix code
 - Create paths from starting state ϵ for each test
- Make state machine re-entrant
 - Create secondary transitions leading to start of other patterns
 - Create tertiary transitions to starting state ϵ
- Allow X output values that prohibit fault propagation
 - Some inputs can produce undefined outputs for defective cell
 - Classical cell-aware fault models exclude those inputs



Cell-Aware Testing

Cell-Aware SBST Generation for RISC-V CLIC

Automatic Cell-Aware Software-Based Self-Test Generation
for a RISC-V Core Local Interrupt Controller

IEEE ATS 2025 – Paper

- Cell-aware test generation has higher runtime
- Cell-aware patterns have lower fault coverage
- Cell-aware fault model has $\approx 10\%$ of untestable faults

Scale⁴Edge

RISC-V®

	Runtime	Stuck-At			Runtime	Cell-Aware		
		Faults	Fault Coverage	Fault Efficiency		Faults	Fault Coverage	Fault Efficiency
CLIC w/o Interface	5.04 h	18,785	83.15 %	83.63 %	35.17 h	87,425	65.56 %	77.14 %
CLIC w/ Interface	3.59 h	18,600	93.25 %	93.57 %	32.93 h	87,797	71.73 %	81.89 %

Cell-Aware Testing

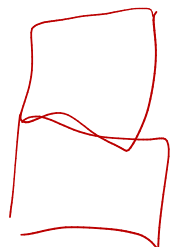
Cell-Aware SBST Generation for RISC-V CLIC

Automatic Cell-Aware Software-Based Self-Test Generation
for a RISC-V Core Local Interrupt Controller

IEEE ATS 2025 – Paper

- Cell-aware patterns detect almost all stuck-at faults
- Stuck-at patterns don't detect all cell-aware faults

	Stuck-At Patterns evaluated on Cell-Aware Fault Model			Cell-Aware Patterns evaluated on Stuck-At Fault Model		
	Runtime	Faults	Fault Coverage	Runtime	Faults	Fault Coverage
CLIC w/o Test Interface	0.64 h	87,425	59.64 %	0.20 h	18,785	82.87 %
CLIC w/ Test Interface	0.85 h	87,797	65.15 %	0.22 h	18,600	92.68 %

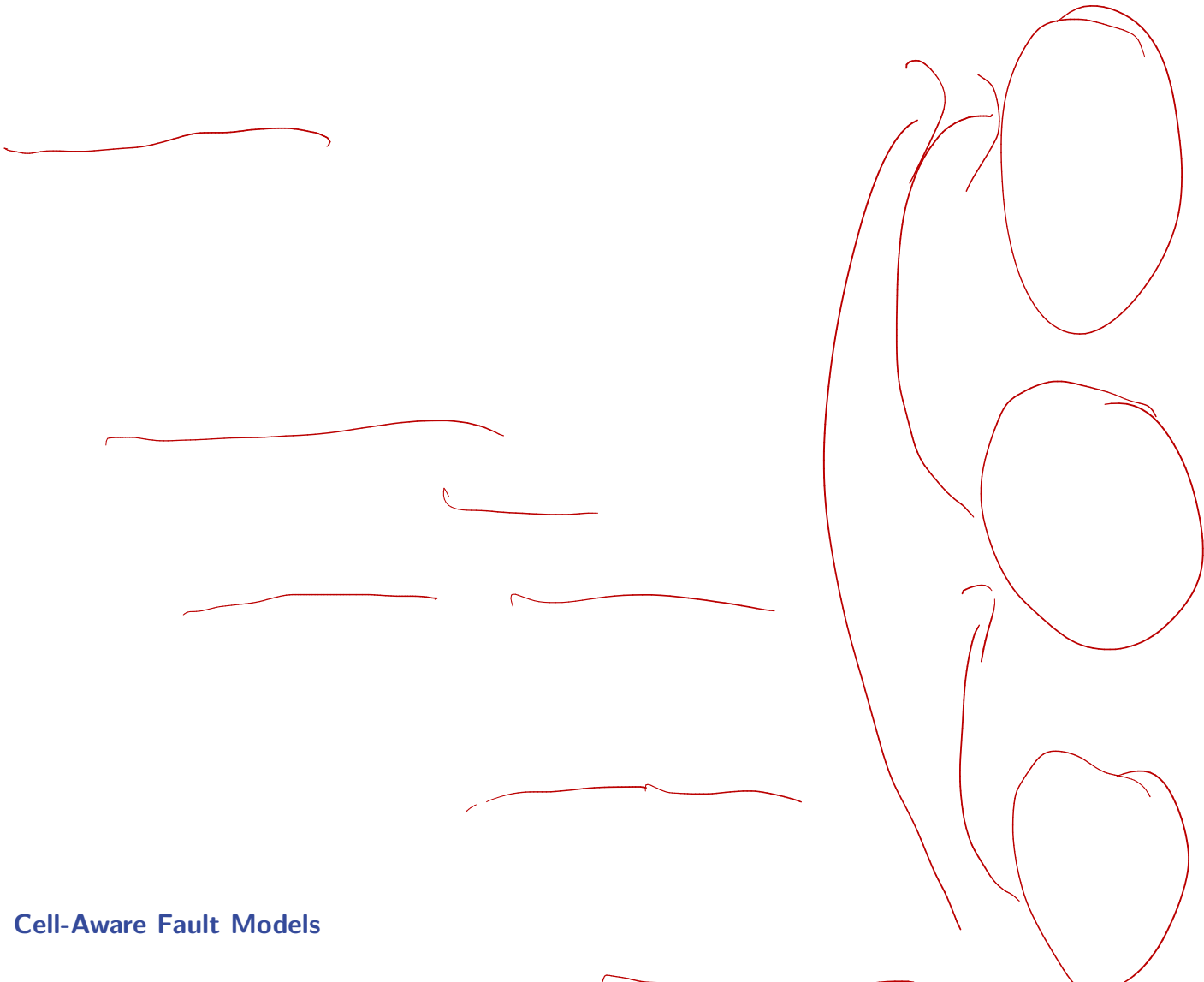
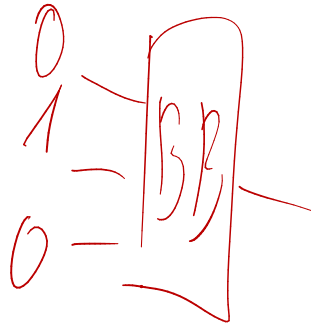


Cell-Aware Testing - References I

- [1] F. Hapke, R. Krenz-Baath, A. Glowatz, et al. “Defect-oriented cell-aware ATPG and fault simulation for industrial cell libraries and designs”. In: *2009 International Test Conference*. 2009, pp. 1–10. DOI: [10.1109/TEST.2009.5355741](https://doi.org/10.1109/TEST.2009.5355741).
- [2] Friedrich Hapke, Wilfried Redemund, Andreas Glowatz, et al. “Cell-Aware Test”. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 33.9 (2014), pp. 1396–1409. DOI: [10.1109/TCAD.2014.2323216](https://doi.org/10.1109/TCAD.2014.2323216).
- [3] Shreyas Pramod Dixit, Divyeshkumar Dhanjibhai Vora, and Ke Peng. “Challenges in Cell-Aware Test”. In: *2018 IEEE 23rd European Test Symposium (ETS)*. 2018, pp. 1–6. DOI: [10.1109/ETS.2018.8400700](https://doi.org/10.1109/ETS.2018.8400700).
- [4] Dave Macemon. *User Defined Fault Models*. Sept. 2011.
- [5] *CMGen User Guide*. 2022.

Cell-Aware Testing - References II

- [6] Tobias Faller and Bernd Becker. "Automatic Cell-Aware Software-Based Self-Test Generation for a RISC-V Core Local Interrupt Controller". In: *2025 IEEE 34th Asian Test Symposium (ATS)*. 2025, pp. 13–18. DOI: [10.1109/ATS66998.2025.00011](https://doi.org/10.1109/ATS66998.2025.00011).



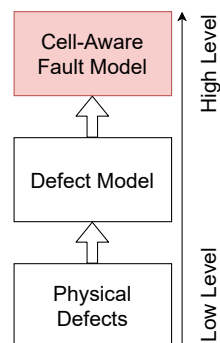
Cell-Aware Fault Models

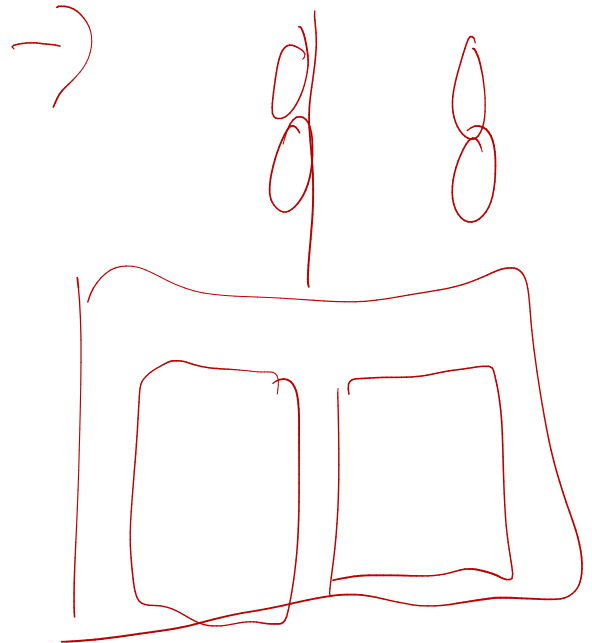
Defect-orient cell-aware fault model creation:

- Derive high-level fault model from low-level physical defects
 - Approximate physical defects with defect model
 - Assume only a single defect exists
 - Assume defect only affects a single cell
 - Derive high-level cell behavior from simulation of defect

⇒ Cell-aware fault model captures cell behavior under defect

- ATPG later applies cell-aware fault model to cells in CUT
- ATPG generates test patterns for modelled low-level physical defects





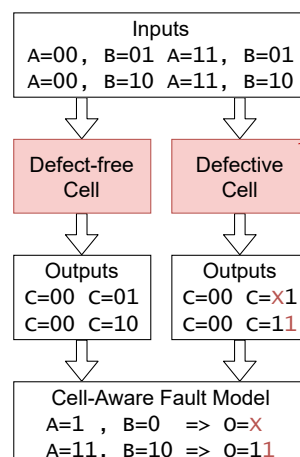
Cell-Aware Fault Models

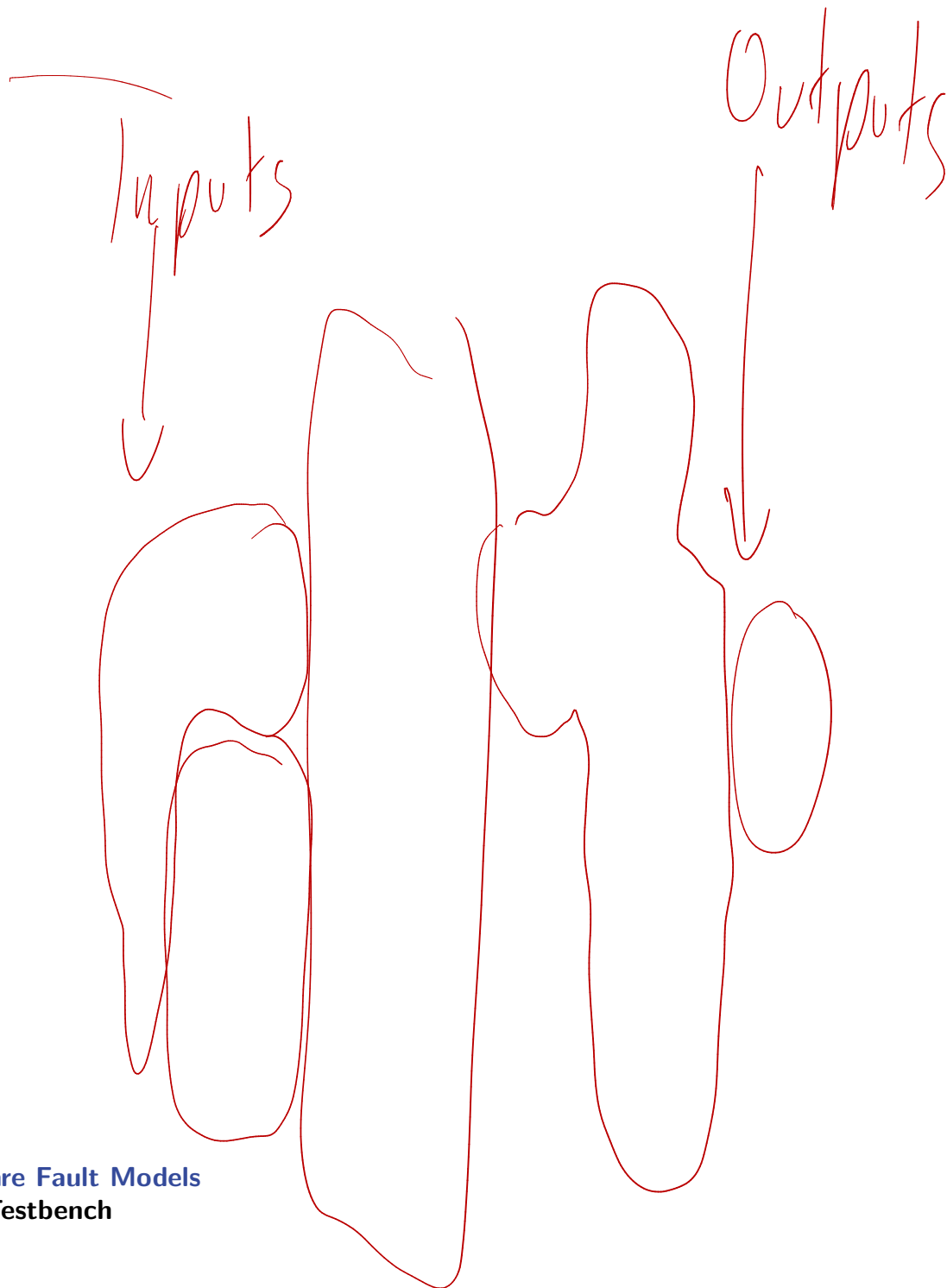
Defect-orient cell-aware fault model creation:

- Simulate possible defects on each cell
 - Model physical defects using resistors
 - Apply defect model to SPICE netlist of cells
- Compare defective cell behavior with defect-free cell
- Store differing behavior in cell-aware fault model

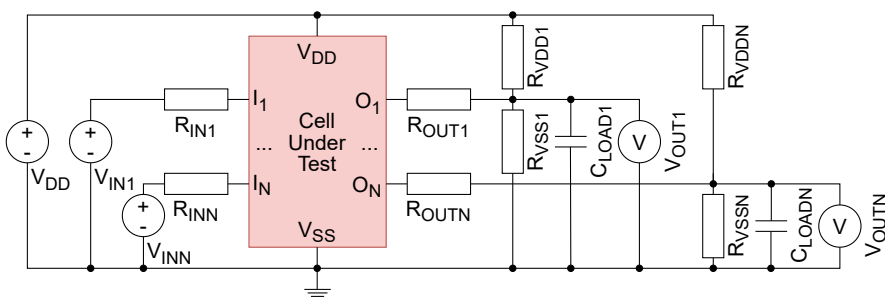
⇒ ATPG later applies cell-aware fault model to CUT

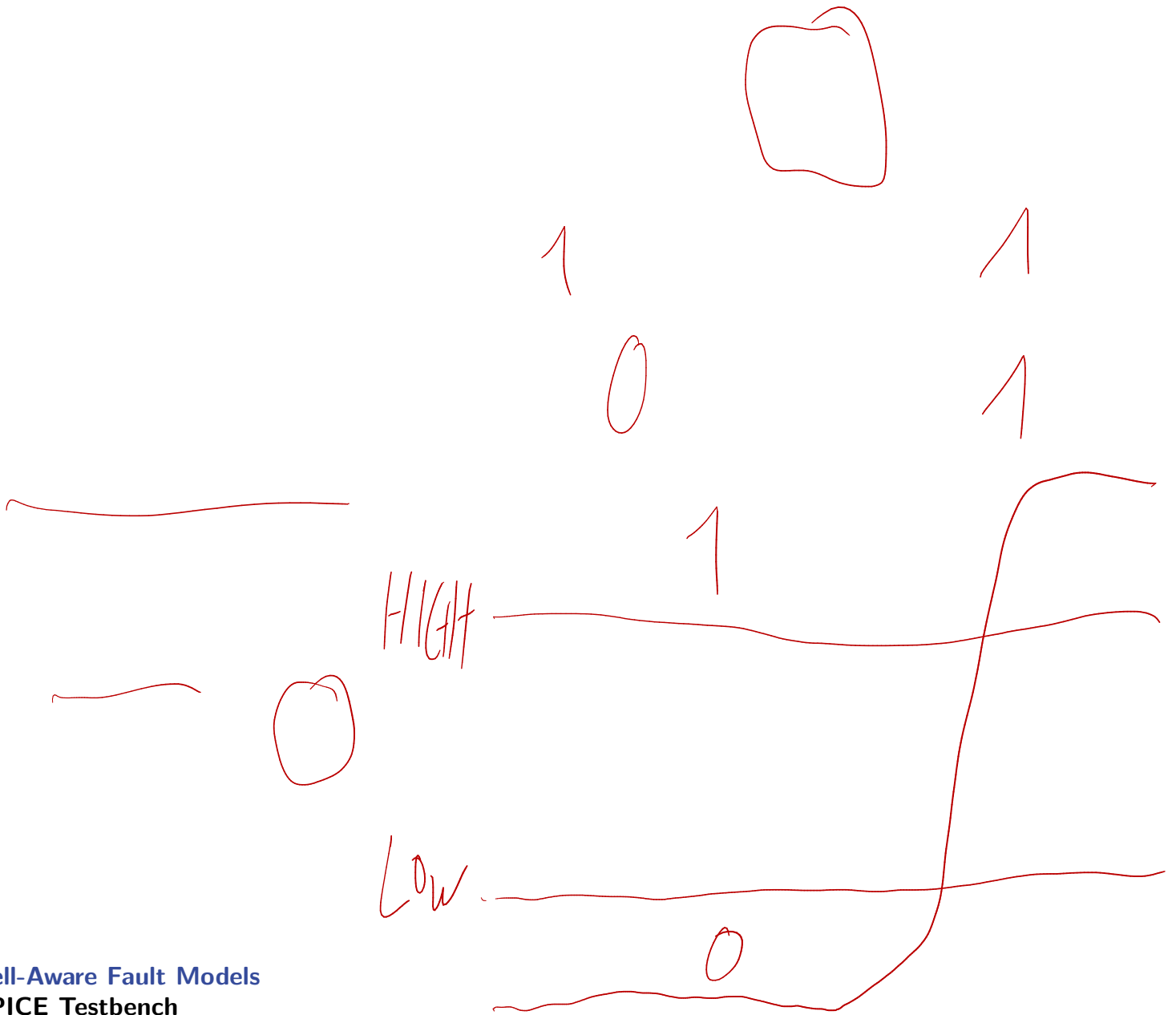
- ATPG justifies inputs given by cell-aware fault model
- ATPG propagates outputs given by cell-aware fault model





Cell-Aware Fault Models SPICE Testbench

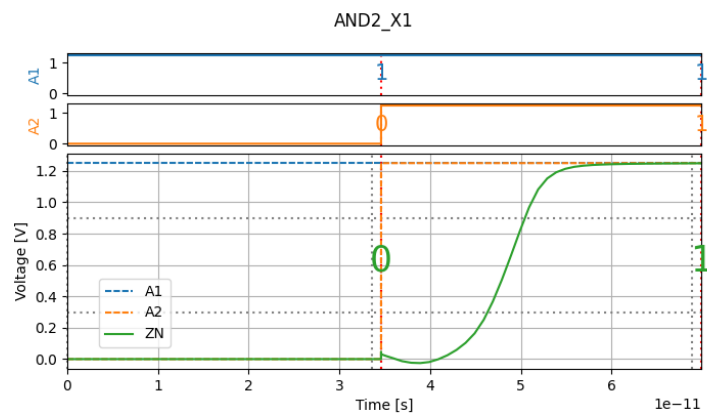




Cell-Aware Fault Models

SPICE Testbench

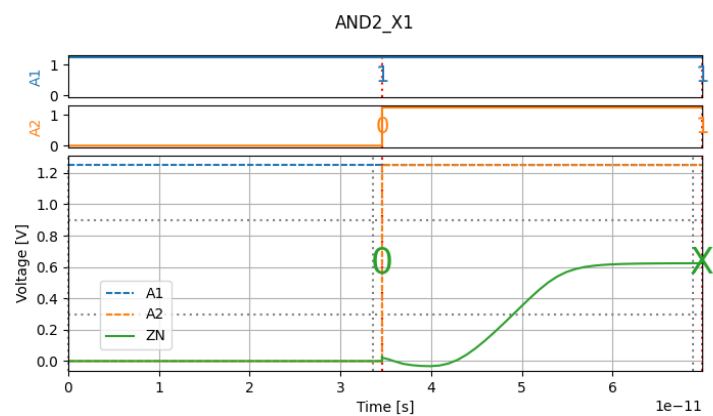
- SPICE simulation is analog
- Digital values are assigned
 - $V \leq \text{LOW} \Rightarrow 0$
 - $\text{LOW} \leq V \leq \text{HIGH} \Rightarrow X$
 - $V \geq \text{HIGH} \Rightarrow 1$

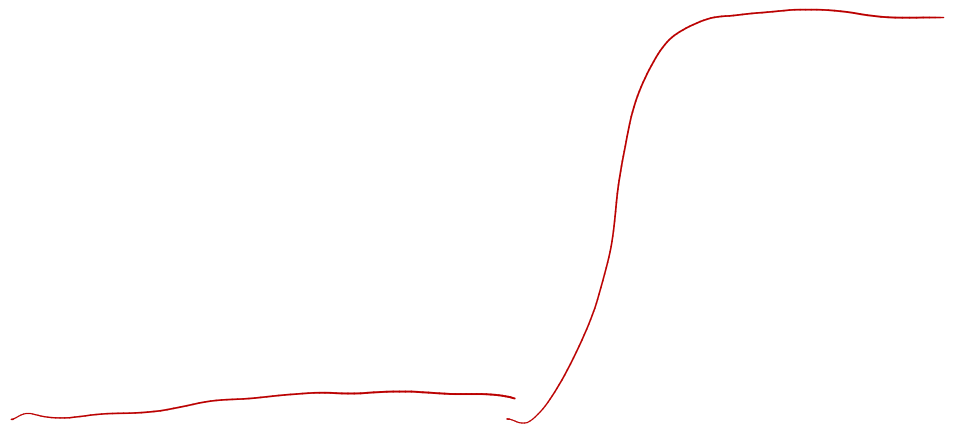


Cell-Aware Fault Models

SPIKE Testbench

- SPICE simulation is analog
- Digital values are assigned
 - $V \leq \text{LOW} \Rightarrow 0$
 - $\text{LOW} \leq V \leq \text{HIGH} \Rightarrow X$
 - $V \geq \text{HIGH} \Rightarrow 1$

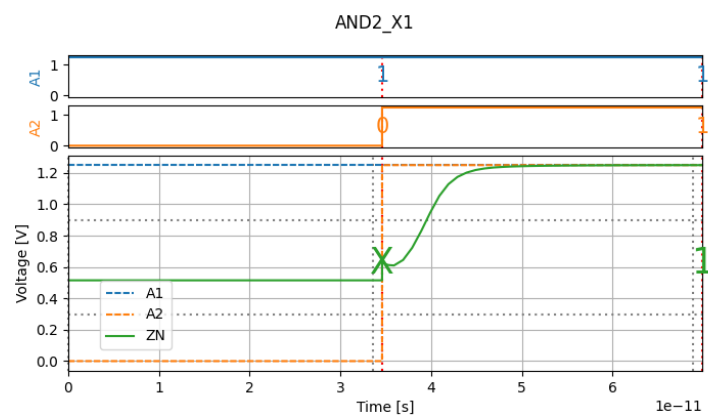




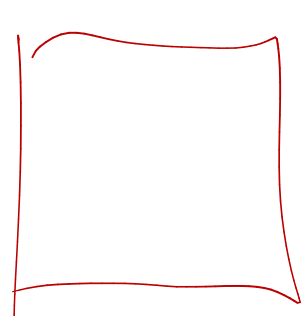
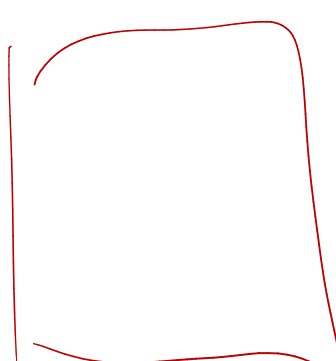
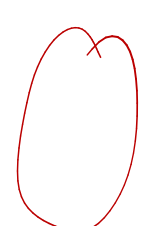
Cell-Aware Fault Models

SPICE Testbench

- SPICE simulation is analog
- Digital values are assigned
 - $V \leq \text{LOW} \Rightarrow 0$
 - $\text{LOW} \leq V \leq \text{HIGH} \Rightarrow X$
 - $V \geq \text{HIGH} \Rightarrow 1$



1/1/1

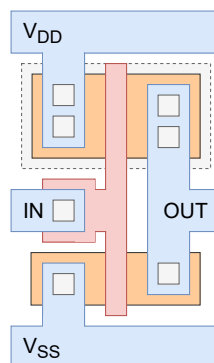
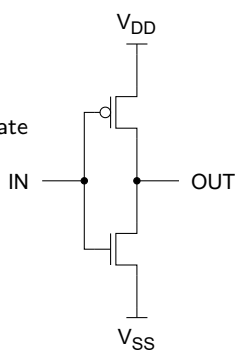


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Defect-free inverter cell:

- Two complementary transistors (CMOS)
 - PMOS (top) : P+-doped contacts, N-doped well
 - NMOS (bottom): N+-doped contacts, P-doped substrate
- Power rails V_{DD} and V_{SS} on metal 1 (blue)
- Transistor gates out polysilicon (red)
- Transistor channels (orange) below gates
- Contacts (gray) connect layers (metal 1 to channel / polysilicon)



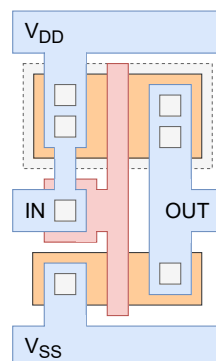
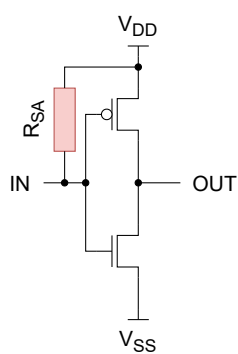


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Stuck-at defects:

- Bridge between port and supply on metal 1 layer
- Very low resistance (e.g., $R_{SA} \approx 0.1 \Omega$)
- Analogous to stuck-at fault model



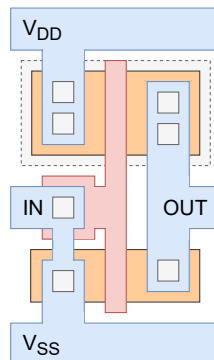
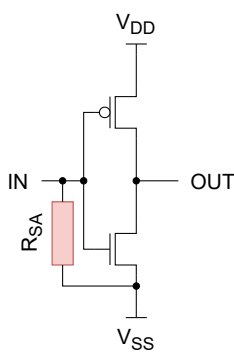


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Stuck-at defects:

- Bridge between port and supply on metal 1 layer
- Very low resistance (e.g., $R_{SA} \approx 0.1 \Omega$)
- Analogous to stuck-at fault model

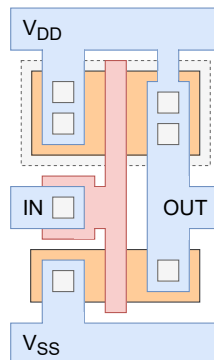
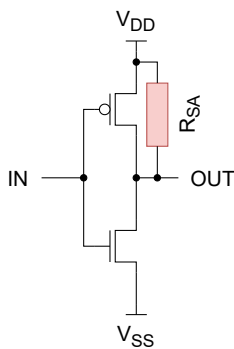


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Stuck-at defects:

- Bridge between port and supply on metal 1 layer
- Very low resistance (e.g., $R_{SA} \approx 0.1 \Omega$)
- Analogous to stuck-at fault model

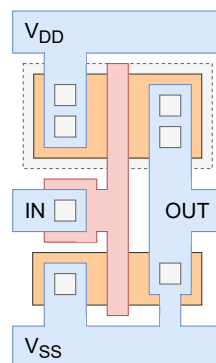
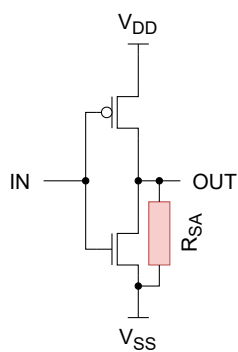


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Stuck-at defects:

- Bridge between port and supply on metal 1 layer
- Very low resistance (e.g., $R_{SA} \approx 0.1 \Omega$)
- Analogous to stuck-at fault model

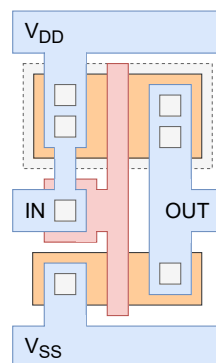
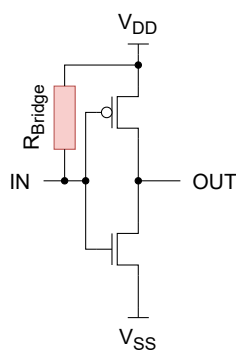


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Bridge defects:

- Bridge between lines on metal 1 layer
- Low resistance (e.g., $R_{\text{Bridge}} \approx 1\ \Omega$)
- $\Theta(n^2)$ theoretical bridges possible
 - Only bridges between adjacent lines are relevant
 - ⇒ Relevant line pairs are derived from layout

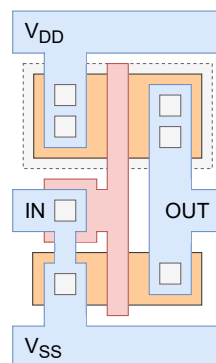
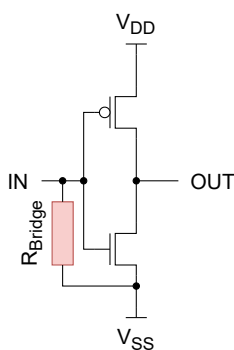


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Bridge defects:

- Bridge between lines on metal 1 layer
- Low resistance (e.g., $R_{\text{Bridge}} \approx 1\ \Omega$)
- $\Theta(n^2)$ theoretical bridges possible
 - Only bridges between adjacent lines are relevant
 - ⇒ Relevant line pairs are derived from layout

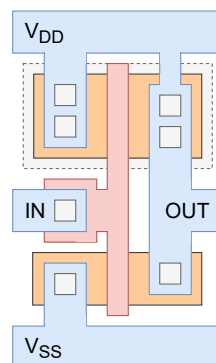
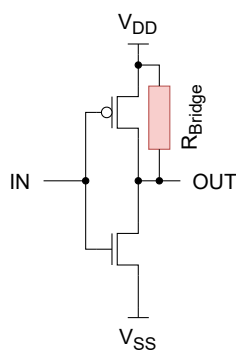


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Bridge defects:

- Bridge between lines on metal 1 layer
- Low resistance (e.g., $R_{\text{Bridge}} \approx 1\ \Omega$)
- $\Theta(n^2)$ theoretical bridges possible
 - Only bridges between adjacent lines are relevant
 - ⇒ Relevant line pairs are derived from layout

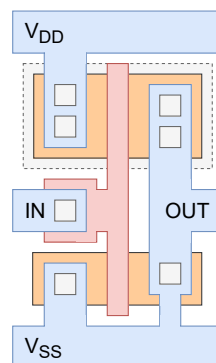
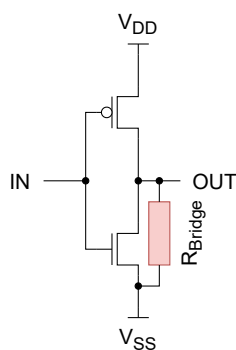


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Bridge defects:

- Bridge between lines on metal 1 layer
- Low resistance (e.g., $R_{\text{Bridge}} \approx 1\ \Omega$)
- $\Theta(n^2)$ theoretical bridges possible
 - Only bridges between adjacent lines are relevant
 - ⇒ Relevant line pairs are derived from layout

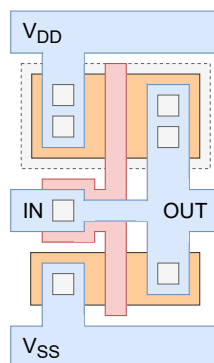
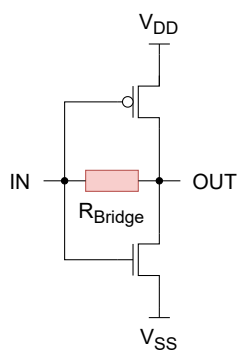


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Bridge defects:

- Bridge between lines on metal 1 layer
- Low resistance (e.g., $R_{\text{Bridge}} \approx 1\ \Omega$)
- $\Theta(n^2)$ theoretical bridges possible
 - Only bridges between adjacent lines are relevant
 - ⇒ Relevant line pairs are derived from layout

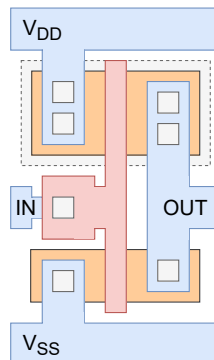
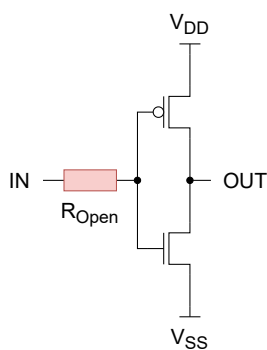


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Open defects:

- Open line section on metal 1 layer
- High resistance (e.g., $R_{\text{Open}} \approx 100 \text{ k}\Omega$)



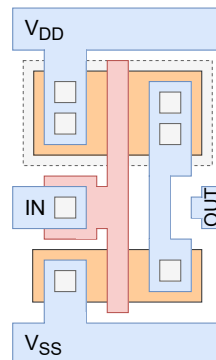
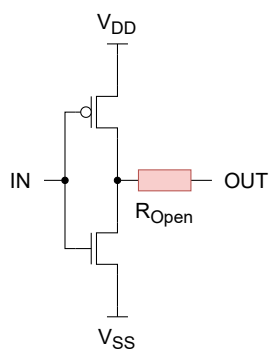


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Open defects:

- Open line section on metal 1 layer
- High resistance (e.g., $R_{\text{Open}} \approx 100 \text{ k}\Omega$)



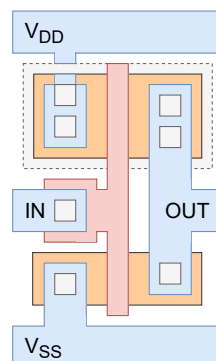
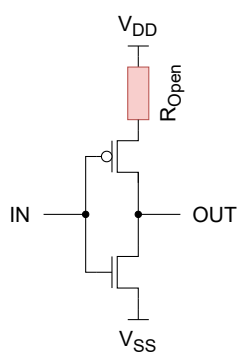


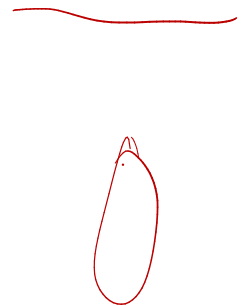
Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Open defects:

- Open line section on metal 1 layer
- High resistance (e.g., $R_{\text{Open}} \approx 100 \text{ k}\Omega$)



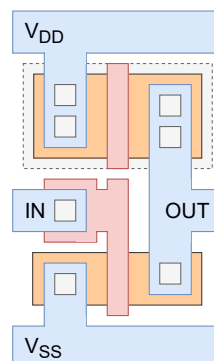
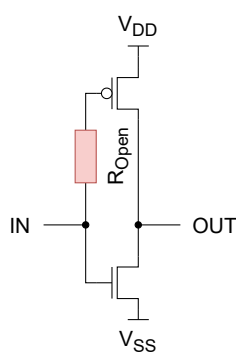


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Open defects:

- Open line section on metal 1 layer
- High resistance (e.g., $R_{Open} \approx 100\text{ k}\Omega$)

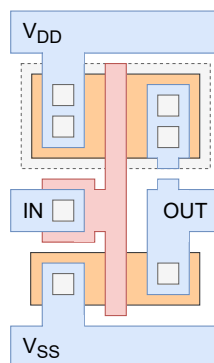
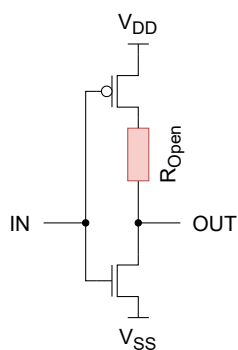


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Open defects:

- Open line section on metal 1 layer
- High resistance (e.g., $R_{\text{Open}} \approx 100 \text{ k}\Omega$)

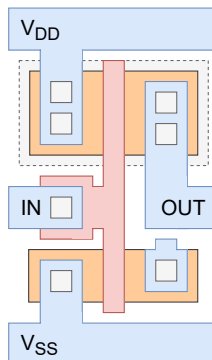
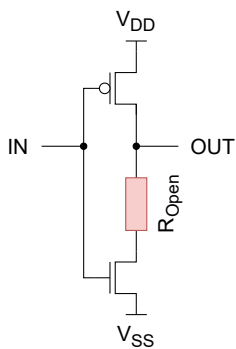


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Open defects:

- Open line section on metal 1 layer
- High resistance (e.g., $R_{\text{Open}} \approx 100 \text{ k}\Omega$)

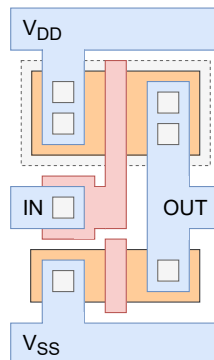
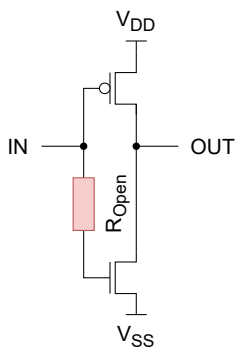


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Open defects:

- Open line section on metal 1 layer
- High resistance (e.g., $R_{\text{Open}} \approx 100 \text{ k}\Omega$)

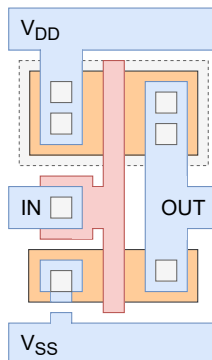
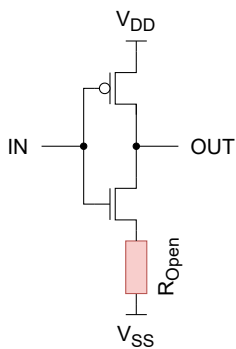


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Open defects:

- Open line section on metal 1 layer
- High resistance (e.g., $R_{\text{Open}} \approx 100 \text{ k}\Omega$)

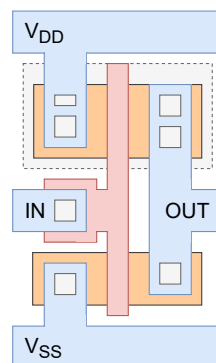
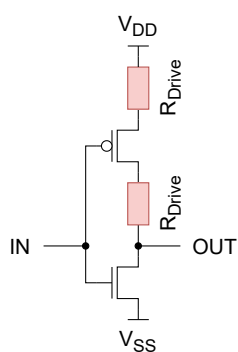


Cell-Aware Fault Models

SPIICE Testbench - Cell Defect Injection

Transistor drive defect:

- Transistor does not conduct well when turned on
- Medium resistance (e.g., $R_{\text{Drive}} \approx 1 \text{ k}\Omega$)

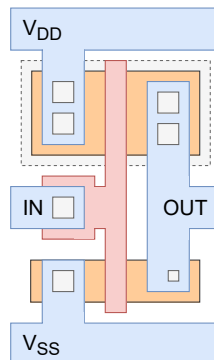
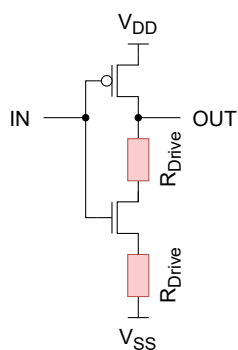


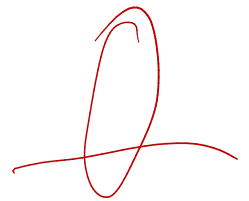
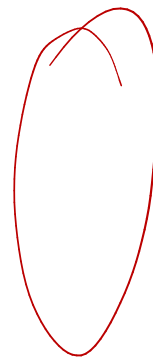
Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Transistor drive defect:

- Transistor does not conduct well when turned on
- Medium resistance (e.g., $R_{\text{Drive}} \approx 1 \text{ k}\Omega$)



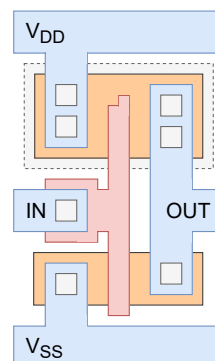
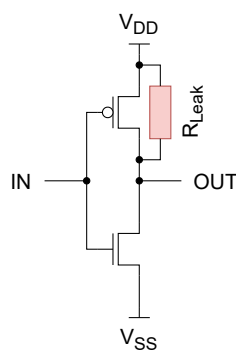


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Transistor leak defect:

- Transistor does not isolate well when turned off
- Low resistance (e.g., $R_{\text{Leak}} \approx 1\ \Omega$)

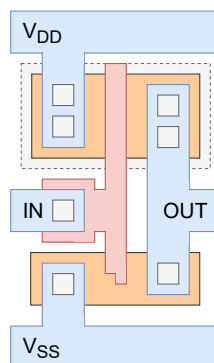
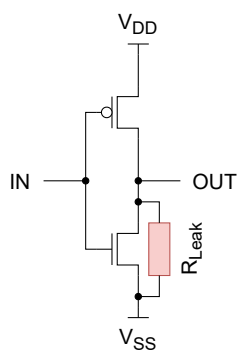


Cell-Aware Fault Models

SPICE Testbench - Cell Defect Injection

Transistor leak defect:

- Transistor does not isolate well when turned off
- Low resistance (e.g., $R_{\text{Leak}} \approx 1\ \Omega$)





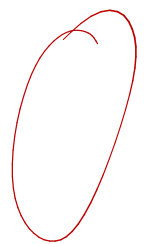
Cell-Aware Fault Models

Example: Cell-Aware Fault Model Generation (Scale4Edge)

Cell-Aware Test for RISC-V

Master Thesis – Markus Schwörer

- PySPICE-based cell-aware fault model generation
- Evaluation on Open-PDK Nangate 45nm cell library
- Parallel simulation on test system
 - AMD Ryzen Threadripper 3970X 32-Core Processor
 - 256 GB DDR4 3600 RAM



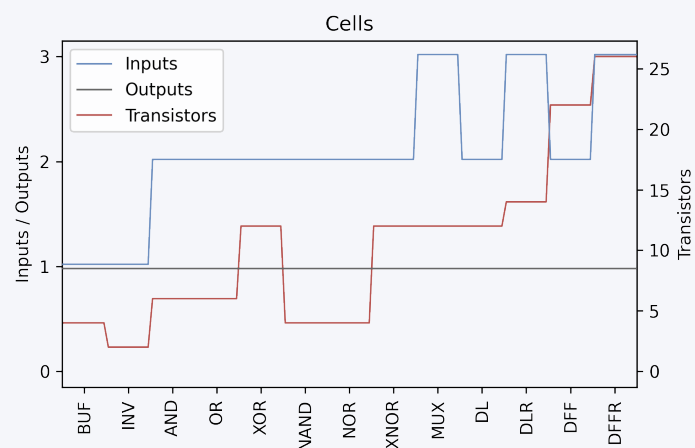
Cell-Aware Fault Models

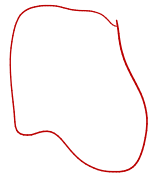
Example: Cell-Aware Fault Model Generation (Scale4Edge) - Generic

Generic cell library:

- Library for ATPG benchmarking
- Minimal set of simple cells
- No underlying technology process

Metric	Library
Combinational cells	9
Sequential cells	4





Cell-Aware Fault Models

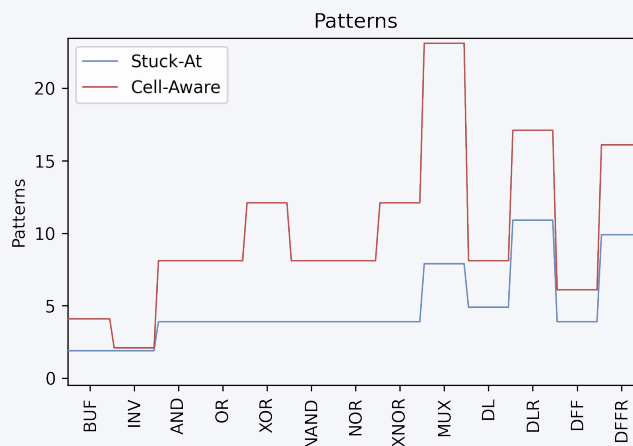
Example: Cell-Aware Fault Model Generation (Scale4Edge) - Generic

SPICE simulations:

- Static inputs (1 TF)
- Transition inputs (2 TF)

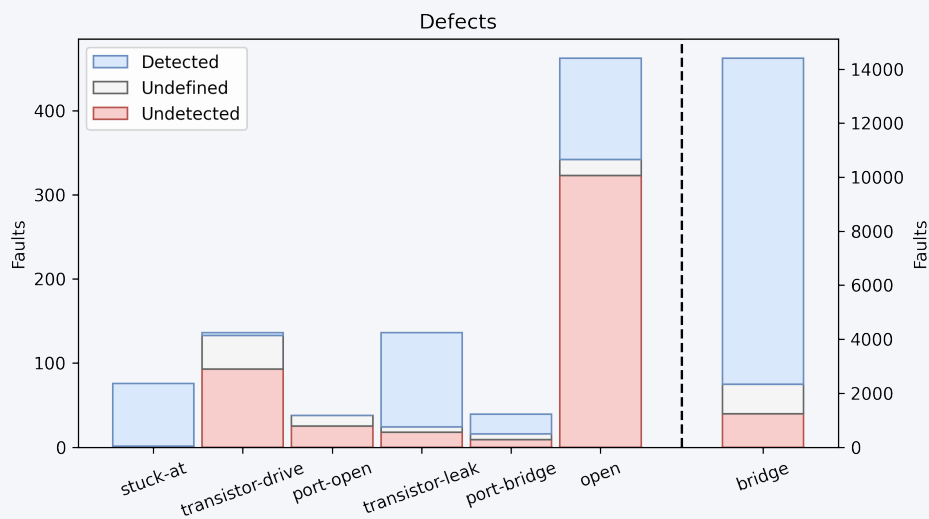
Metric	Result
SPICE simulations	396,796
Simulation time	4.68 h
Analysis time	15.04 s

- Cell-aware defects require more patterns
- ⇒ Stuck-at tests miss cell-aware defects



Cell-Aware Fault Models

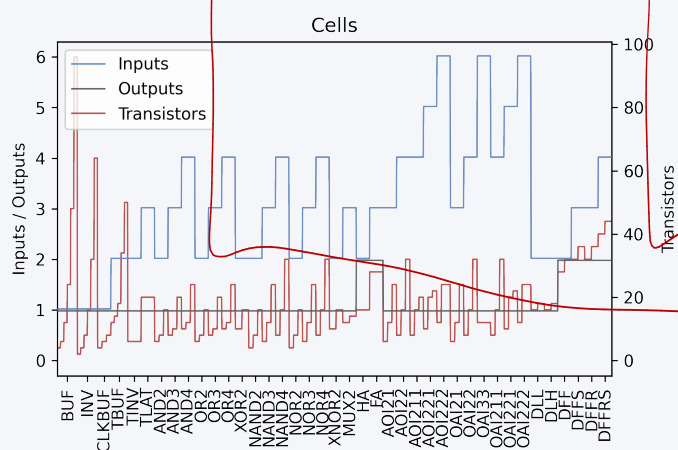
Example: Cell-Aware Fault Model Generation (Scale4Edge) - Generic



Example: Cell-Aware Fault Model Generation (Scale4Edge) - Nangate45

- Academic semi-realistic technology
- Many variants for each cell type

Metric	Library
Combinational cells	105
Sequential cells	21



Cell-Aware Fault Models

Example: Cell-Aware Fault Model Generation (Scale4Edge) - Nangate45

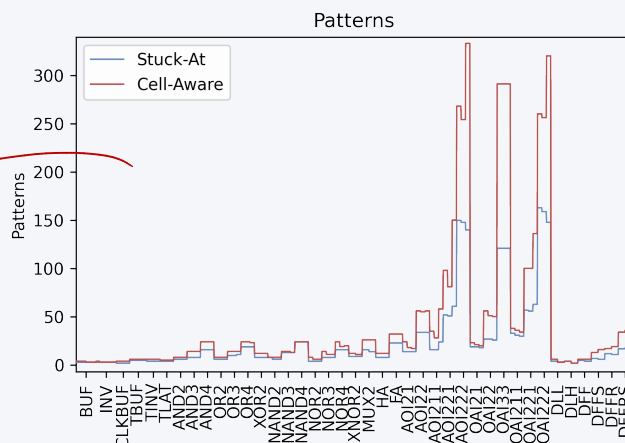
SPICE simulations:

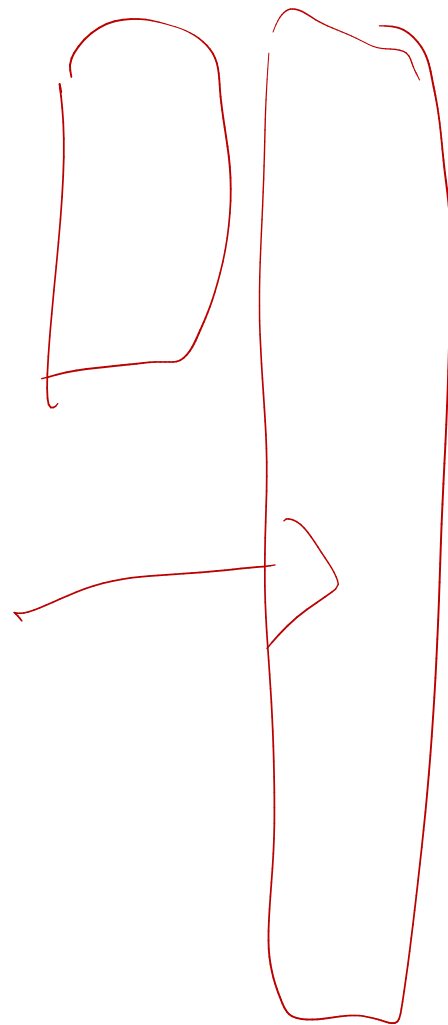
- Static inputs (1 TF)
- Transition inputs (2 TF)

Metric	Result
SPICE simulations	40,605,573
Simulation time	6.49 d
Analysis time	23.10 min

- Cell-aware defects require more patterns

⇒ Stuck-at tests miss cell-aware defects





Cell-Aware Fault Models

Example: Cell-Aware Fault Model Generation (Scale4Edge) - Nangate45



Cell-Aware Fault Models - References I

- [1] F. Hapke, R. Krenz-Baath, A. Glowatz, et al. "Defect-oriented cell-aware ATPG and fault simulation for industrial cell libraries and designs". In: *2009 International Test Conference*. 2009, pp. 1–10. DOI: 10.1109/TEST.2009.5355741.
- [2] Friedrich Hapke, Wilfried Redemund, Andreas Glowatz, et al. "Cell-Aware Test". In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 33.9 (2014), pp. 1396–1409. DOI: 10.1109/TCAD.2014.2323216.
- [3] Friedrich Hapke, Will Howell, Peter Maxwell, et al. "Defect-Oriented Test: Effectiveness in High Volume Manufacturing". In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 40.3 (2021), pp. 584–597. DOI: 10.1109/TCAD.2020.3001259.
- [4] Sujay Pandey, Sanya Gupta, Madhu Sudhan L., et al. "Characterization of Library Cells for Open-circuit Defect Exposure: A Systematic Methodology". In: *2019 IEEE International Test Conference (ITC)*. 2019, pp. 1–10. DOI: 10.1109/ITC44170.2019.9000154.

Cell-Aware Fault Models - References II

- [5] Suriyaprakash Natarajan, Abhijit Sathaye, Chaitali Oak, et al. “DEFCON: Defect Acceleration through Content Optimization”. In: *2022 IEEE International Test Conference (ITC)*. 2022, pp. 298–304. DOI: 10.1109/ITC50671.2022.00038.
- [6] Xhesila Xhafa, Eric Faehn, Patrick Girard, et al. “SRAM Periphery Testing Using the Cell-Aware Test Methodology”. In: *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44.5 (2025), pp. 2000–2013. DOI: 10.1109/TCAD.2024.3506854.

